Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп’ютерних технологій
Кафедра оптоелектроніки та інформаційних технологій

Звіт
До лабораторної роботи №6
Варіант 14
На тему
«Замикання та декоратори як HOF»

Виконав:
Студент групи ФЕП-21с
Хороший Святослав

Перевірив:
Ас. Остріков Д. А.

Львів 2026

Мета роботи

Навчитися створювати замикання зі станом і параметризовані декоратори як функції вищого порядку. Реалізувати політики повторів задач, ін’єкцію залежностей і детерміновані тести без реальних затримок.

Теоретичні відомості

Замикання зберігає доступ до змінних зовнішньої функції після її завершення. Nonlocal дозволяє змінювати захоплений стан. Фабрика декоратора приймає налаштування, повертає декоратор, а той — обгортку функції.

Functools.wraps зберігає ім’я, документацію та __wrapped__. ParamSpec і TypeVar передають типи аргументів та результату. Залежності clock, sleep, logger і time_fn передаються функціями, щоб замінювати зовнішні ефекти під час тестування.

Хід роботи

Для варіанта 14 я реалізував планувальник повторів задач. У closures.py створив make_counter зі змінним станом через nonlocal, make_backoff із політикою затримок та make_logger із префіксом. Фіксована політика використовує factor=1, експоненційна — factor>1.

Декоратор retries приймає delay_policy як Callable. Після невдалої спроби і пауза дорівнює delay × factor^(i−1). Після останньої спроби паузи немає: піднімається початковий виняток. Винятки поза налаштованим переліком поширюються одразу, некоректна пауза відхиляється до sleep.

У timed використано finally для логування успіху та помилки. Додатково реалізовано cached із TTL, cache_clear і cache_invalidate. Ключ враховує args і kwargs; успішний результат зберігається після завершення функції. На точній межі TTL потрібне нове обчислення. Методи кешу описано типізованим Protocol.

Тести порівнюють два порядки композиції. Зовнішній timed вимірює весь цикл retries, а timed усередині retries — кожну спробу. Перевірено метадані функцій, незалежність лічильників, успіх, винятки, межі параметрів і керування кешем.

У main.py задача успішна з четвертої спроби. Для delay=0,5 фіксовані паузи становлять [0,5; 0,5; 0,5], експоненційні з factor=2 — [0,5; 1; 2]. Fake_sleep записує паузи у список. Демонстраційний clock повертає 0 та 0,012: надруковані 12 мс є заданим прикладом, а не виміром швидкодії.

lab06 / closures.py

```python
"""Замикання для стану, політики затримок і логування."""

from collections.abc import Callable
from math import isfinite


def make_counter(start: int = 0) -> Callable[[], int]:
    value = start

    def increment() -> int:
        nonlocal value
        value += 1
        return value

    return increment


def make_backoff(*, delay: float, factor: float = 1.0) -> Callable[[int], float]:
    """Номер невдалої спроби починається з 1; factor=1 дає фіксовану паузу."""
    if not isfinite(delay) or delay < 0 or not isfinite(factor) or factor < 1:
        raise ValueError("delay >= 0 і factor >= 1 мають бути скінченними")

    def policy(failed_attempt: int) -> float:
        if failed_attempt < 1:
            raise ValueError("Номер спроби має починатися з 1")
        return delay * factor ** (failed_attempt - 1)

    return policy


def make_logger(prefix: str, sink: Callable[[str], None]) -> Callable[[str], None]:
    def log(message: str) -> None:
        sink(f"[{prefix}] {message}")

    return log
```

Рис. 1. Файл closures.py, рядки 1–35

lab06 / decorators.py

```python
"""Параметризовані HOF з ін'єкцією часу, політики, sleep і logger."""

from collections.abc import Callable
from functools import wraps
from math import isfinite
from threading import Lock
from time import monotonic, perf_counter
from time import sleep as real_sleep
from typing import Literal, ParamSpec, Protocol, TypeVar, cast

P = ParamSpec("P")
R = TypeVar("R")
Key = tuple[tuple[object, ...], tuple[tuple[str, object], ...]]
Predicate = Callable[[tuple[object, ...], dict[str, object]], bool]


def timed(
    label: str | None = None,
    *,
    unit: Literal["s", "ms", "us"] = "ms",
    logger: Callable[[str], None],
    clock: Callable[[], float] = perf_counter,
) -> Callable[[Callable[P, R]], Callable[P, R]]:
    factor = {"s": 1.0, "ms": 1000.0, "us": 1_000_000.0}[unit]

    def decorate(func: Callable[P, R]) -> Callable[P, R]:
        @wraps(func)
        def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
            start = clock()
            try:
                return func(*args, **kwargs)
            finally:
```

Рис. 2. Файл decorators.py, рядки 1–32

lab06 / decorators.py

```
33                 elapsed = (clock() - start) * factor
34                 logger(f"[timed] {label or func.__name__}: {elapsed:.3f} {unit}")
35
36         return wrapper
37
38     return decorate
39
40
41 def retries(
42     *,
43     attempts: int,
44     delay_policy: Callable[[int], float],
45     exceptions: tuple[type[Exception], ...] = (TimeoutError,),
46     sleep: Callable[[float], None] = real_sleep,
47     logger: Callable[[str], None],
48 ) -> Callable[[Callable[P, R]], Callable[P, R]]:
49     if attempts < 1 or not exceptions:
50         raise ValueError("attempts >= 1; exceptions має бути непорожнім")
51
52     def decorate(func: Callable[P, R]) -> Callable[P, R]:
53         @wraps(func)
54         def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
55             for attempt in range(1, attempts + 1):
56                 try:
57                     return func(*args, **kwargs)
58                 except exceptions:
59                     if attempt == attempts:
60                         logger(f"[retries] вичерпано {attempt} спроб")
61                         raise
62                     wait = delay_policy(attempt)
63                     if not isfinite(wait) or wait < 0:
64                         raise ValueError("Політика має повертати скінченну паузу >= 0")
```

Рис. 3. Файл decorators.py, рядки 33–64

lab06 / decorators.py

```
65                     logger(f"[retries] спроба {attempt}, пауза {wait:.3f} s")
66                     sleep(wait)
67             raise AssertionError("Недосяжний код")
68
69         return wrapper
70
71     return decorate
72
73
74 class Cached(Protocol[P, R]):
75     __name__: str
76     __doc__: str | None
77     __wrapped__: Callable[P, R]
78
79     def __call__(self, *args: P.args, **kwargs: P.kwargs) -> R: ...
80     def cache_clear(self) -> None: ...
81     def cache_invalidate(self, predicate: Predicate) -> int: ...
82
83
84 def cached(
85     *,
86     ttl: float | None = None,
87     time_fn: Callable[[], float] = monotonic,
88 ) -> Callable[[Callable[P, R]], Cached[P, R]]:
89     if ttl is not None and (not isfinite(ttl) or ttl < 0):
90         raise ValueError("TTL має бути скінченним >= 0 або None")
91
92     def decorate(func: Callable[P, R]) -> Cached[P, R]:
93         cache: dict[Key, tuple[float | None, R]] = {}
94         lock = Lock()
95
96         @wraps(func)
```

Рис. 4. Файл decorators.py, рядки 65–96

lab06 / decorators.py

```
97          def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
98              key: Key = (args, tuple(sorted(kwargs.items())))
99              hash(key)  # Явно відхиляємо нехешовані аргументи.
100             with lock:
101                 entry = cache.get(key)
102                 if entry is not None:
103                     expiry, value = entry
104                     if expiry is None or time_fn() < expiry:
105                         return value
106                     del cache[key]
107             value = func(*args, **kwargs)
108             expiry = None if ttl is None else time_fn() + ttl
109             with lock:
110                 cache[key] = (expiry, value)
111             return value
112
113         def clear() -> None:
114             with lock:
115                 cache.clear()
116
117         def invalidate(predicate: Predicate) -> int:
118             with lock:
119                 keys = [key for key in cache if predicate(key[0], dict(key[1]))]
120                 for key in keys:
121                     del cache[key]
122                 return len(keys)
123
124         setattr(wrapper, "cache_clear", clear)
125         setattr(wrapper, "cache_invalidate", invalidate)
126         return cast(Cached[P, R], wrapper)
127
128     return decorate
```

Рис. 5. Файл decorators.py, рядки 97–128

lab06 / main.py

```
1  """Демонстрація планувальника без реального очікування."""
2
3  from closures import make_backoff, make_counter, make_logger
4  from decorators import cached, retries, timed
5
6
7  def main() -> None:
8      print("=== ПЛАНУВАЛЬНИК ПОВТОРІВ | ВАРІАНТ 14 ===")
9      log = make_logger("TASK", print)
10     for name, factor in [("Фіксована", 1.0), ("Експоненційна", 2.0)]:
11         counter = make_counter()
12         waits: list[float] = []
13         ticks = iter([0.0, 0.012])
14
15         @timed(name, logger=log, clock=lambda: next(ticks))
16         @retries(
17             attempts=4,
18             delay_policy=make_backoff(delay=0.5, factor=factor),
19             sleep=waits.append,
20             logger=log,
21         )
22         def task() -> str:
23             if counter() < 4:
24                 raise TimeoutError("Тимчасова помилка задачі")
```

Рис. 6. Файл main.py, рядки 1–24

lab06 / main.py

```
            return "готово"

        print(name, "політика:", task())
        print("Заплановані затримки:", waits)

    calls = make_counter()
    now = 0.0

    @cached(ttl=1, time_fn=lambda: now)
    def result(task_id: int) -> str:
        return f"задача {task_id}, обчислення {calls()}"

    print("Перший виклик:", result(14))
    print("Кеш-хіт:", result(14))
    now = 1.0
    print("Межа TTL:", result(14))
    print("Інвалідація:", result.cache_invalidate(lambda args, kw: args == (14,)))
    print("Після інвалідації:", result(14))
    result.cache_clear()
    print("Після очищення:", result(14))


if __name__ == "__main__":
    main()
```

Рис. 7. Файл main.py, рядки 25–48

lab06 / tests/test_closures.py

```
import pytest

from closures import make_backoff, make_counter, make_logger


def test_counter_state_and_independence() -> None:
    a, b = make_counter(10), make_counter()
    assert [a(), a(), b(), a()] == [11, 12, 1, 13]


def test_fixed_and_exponential() -> None:
    fixed = make_backoff(delay=0.5)
    exponential = make_backoff(delay=0.5, factor=2)
    assert [fixed(i) for i in (1, 2, 3)] == [0.5, 0.5, 0.5]
    assert [exponential(i) for i in (1, 2, 3)] == [0.5, 1, 2]
    with pytest.raises(ValueError):
        fixed(0)


@pytest.mark.parametrize("delay,factor", [(-1, 1), (1, 0.5), (float("inf"), 1)])
def test_invalid_policy(delay: float, factor: float) -> None:
    with pytest.raises(ValueError):
        make_backoff(delay=delay, factor=factor)


def test_logger_closure() -> None:
    logs: list[str] = []
    make_logger("TASK", logs.append)("ready")
    assert logs == ["[TASK] ready"]
```

Рис. 8. Файл tests/test_closures.py, рядки 1–29

lab06 / tests/test_decorators.py

```python
import inspect

import pytest

from closures import make_backoff
from decorators import cached, retries, timed


def test_timed_success_and_metadata() -> None:
    logs: list[str] = []
    ticks = iter([0.0, 0.012345])

    def work(x: int, *, extra: int = 1) -> int:
        """Обчислення."""
        return x + extra

    wrapped = timed("work", logger=logs.append, clock=lambda: next(ticks))(work)
    assert wrapped(4, extra=2) == 6
    assert logs == ["[timed] work: 12.345 ms"]
    assert wrapped.__name__ == work.__name__ and wrapped.__doc__ == work.__doc__
    assert inspect.unwrap(wrapped) is work


def test_composition_counts_each_attempt() -> None:
    logs: list[str] = []
    waits: list[float] = []
    calls = 0

    def work() -> int:
        """Нестабільна задача."""
        nonlocal calls
        calls += 1
        if calls < 3:
            raise TimeoutError("retry")
```

Рис. 9. Файл tests/test_decorators.py, рядки 1–34

lab06 / tests/test_decorators.py

```python
        return 14

    ticks = iter([0.0, 0.001, 1.0, 1.001, 2.0, 2.001])
    measured = timed(logger=logs.append, clock=lambda: next(ticks))(work)
    wrapped = retries(
        attempts=3,
        delay_policy=make_backoff(delay=1, factor=2),
        sleep=waits.append,
        logger=logs.append,
    )(measured)
    assert wrapped() == 14 and calls == 3 and waits == [1, 2]
    assert sum("[timed]" in line for line in logs) == 3
    assert wrapped.__name__ == work.__name__ and wrapped.__doc__ == work.__doc__
    assert inspect.unwrap(wrapped) is work


def test_timed_outside_retries_measures_whole_task() -> None:
    calls = 0
    logs: list[str] = []

    def task() -> str:
        nonlocal calls
        calls += 1
        if calls == 1:
            raise TimeoutError()
        return "ok"

    retry = retries(
        attempts=2, delay_policy=lambda i: 0, sleep=lambda d: None, logger=logs.append
    )(task)
    ticks = iter([0.0, 1.0])
    wrapped = timed(logger=logs.append, clock=lambda: next(ticks))(retry)
    assert wrapped() == "ok"
    assert sum("[timed]" in line for line in logs) == 1
```

Рис. 10. Файл tests/test_decorators.py, рядки 35–68

lab06 / tests/test_decorators.py

```
 69
 70
 71  def test_exhaustion_and_unmatched_exception() -> None:
 72      waits: list[float] = []
 73      original = TimeoutError("original")
 74
 75      def fail() -> None:
 76          raise original
 77
 78      wrapped = retries(
 79          attempts=2, delay_policy=lambda i: 0, sleep=waits.append, logger=lambda s: None
 80      )(fail)
 81      with pytest.raises(TimeoutError) as caught:
 82          wrapped()
 83      assert caught.value is original and waits == [0]
 84      unmatched = retries(
 85          attempts=3,
 86          exceptions=(ValueError,),
 87          delay_policy=lambda i: 0,
 88          sleep=waits.append,
 89          logger=lambda s: None,
 90      )(fail)
 91      with pytest.raises(TimeoutError):
 92          unmatched()
 93      assert waits == [0]
 94
 95
 96  def test_one_attempt_and_invalid_parameters() -> None:
 97      with pytest.raises(ValueError):
 98          retries(attempts=0, delay_policy=lambda i: 0, logger=lambda s: None)
 99      waits: list[float] = []
100
101      def fail() -> None:
102          raise TimeoutError()
```

Рис. 11. Файл tests/test_decorators.py, рядки 69–102

lab06 / tests/test_decorators.py

```
103
104      wrapped = retries(
105          attempts=1, delay_policy=lambda i: 0, sleep=waits.append, logger=lambda s: None
106      )(fail)
107      with pytest.raises(TimeoutError):
108          wrapped()
109      assert waits == []
110      invalid = retries(
111          attempts=2, delay_policy=lambda i: -1, sleep=waits.append, logger=lambda s: None
112      )(fail)
113      with pytest.raises(ValueError):
114          invalid()
115      assert waits == []
116
117
118  def test_cache_ttl_invalidation_and_metadata() -> None:
119      now, calls = 0.0, 0
120
121      def compute(x: int, *, extra: int = 0) -> int:
122          """Результат задачі."""
123          nonlocal calls
124          calls += 1
125          return x + extra
126
127      wrapped = cached(ttl=1, time_fn=lambda: now)(compute)
128      assert wrapped(2, extra=3) == wrapped(2, extra=3) == 5
129      assert calls == 1
130      now = 1.0
131      assert wrapped(2, extra=3) == 5 and calls == 2
132      wrapped(9)
133      assert wrapped.cache_invalidate(lambda a, kw: a == (2,) and kw == {"extra": 3}) == 1
134      assert wrapped.cache_invalidate(lambda a, kw: a == (2,)) == 0
135      wrapped(9)
136      assert calls == 3
```

Рис. 12. Файл tests/test_decorators.py, рядки 103–136

lab06 / tests/test_decorators.py

```
137      wrapped.cache_clear()
138      wrapped(9)
139      assert calls == 4
140      assert wrapped.__name__ == compute.__name__ and wrapped.__doc__ == compute.__doc__
141      assert wrapped.__wrapped__ is compute
142
143
144  @pytest.mark.parametrize("ttl", [-1, float("nan"), float("inf")])
145  def test_invalid_ttl(ttl: float) -> None:
146      with pytest.raises(ValueError):
147          cached(ttl=ttl)
148
149
150  def test_zero_ttl_and_exceptions_not_cached() -> None:
151      calls = 0
152
153      @cached(ttl=0, time_fn=lambda: 0.0)
154      def work() -> int:
155          nonlocal calls
156          calls += 1
157          return calls
158
159      assert work() == 1 and work() == 2
160
161      @cached()
162      def fail() -> None:
163          raise ValueError("not stored")
164
165      for _ in range(2):
166          with pytest.raises(ValueError):
167              fail()
```

Рис. 13. Файл tests/test_decorators.py, рядки 137–167

Запуск і результати перевірки

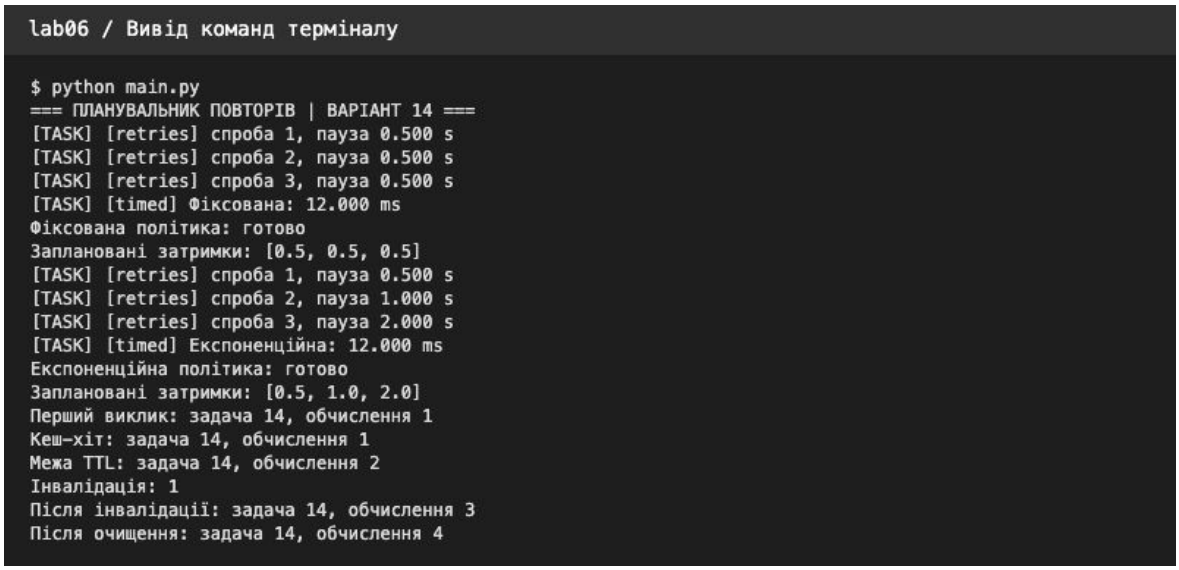


Рис. 14. Запуск демонстрації

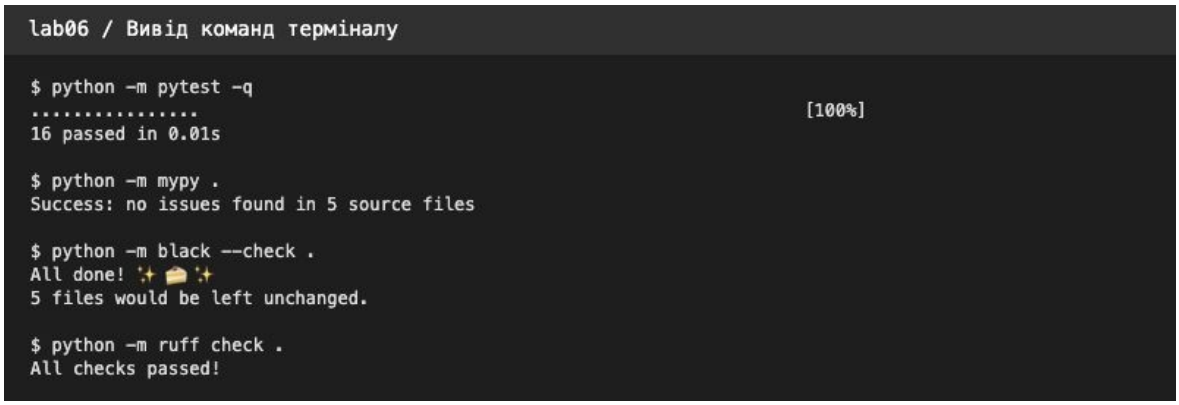


Рис. 15. Тести та перевірки якості

Висновок

Я реалізував замикання для стану, політики затримок і логування та три параметризовані декоратори. Перевірив фіксований і експоненційний backoff, порядок композиції та TTL-кеш. Усі 16 тестів пройшли; mypy, black і ruff помилок не виявили.

Ін’єкція залежностей дозволяє перевірити повтори без очікування й мережі. Wraps зберігає метадані, а ParamSpec і TypeVar — типи виклику. Повторні виклики мають бути допустимими для задачі; сам декоратор retries не забезпечує ідемпотентності.

Роботу виконано за наданим Lab_06.docx, індивідуальний варіант 14.